

Assignment 4

Triangle Rasterization and Depth Buffering

Computer Graphics Course

C++ | CLion | MiniFB | MicroUI | GLM

Student 1	Natalie Bittar ID: 212586044
Student 2	Adan Awad ID:324302678
Course	Computer Graphics
Assignment	Triangle Rasterization and Depth Buffering

This report documents the design, implementation, testing, and results of the software triangle rasterizer developed for Assignment 4. The application fills projected triangles using barycentric coordinates, resolves visibility with a Z-buffer, visualizes depth values, and implements the pair-programming extensions Backface Culling and the Top-Left Fill Rule.

Report Structure

- Part 1: Bounding Box Rasterization
- Part 2: Barycentric Triangle Filling
- Part 3: Z-Buffer and Depth Visualization
- Part 4: Pair Programming Extensions
- Interaction and Testing
- Conclusion

Overview

Assignment 4 converts the previous transparent wireframe viewer into a software rasterizer that draws solid geometry. Each triangular face is projected into screen space, bounded by a two-dimensional rectangle, tested pixel by pixel using barycentric coordinates, and written into the color framebuffer. A separate floating-point Z-buffer stores the nearest depth drawn at every pixel, ensuring that front geometry correctly hides geometry behind it.

The application retains the three OBJ models used in the previous assignments: a cube, pyramid, and octahedron. Every face receives a deterministic pseudo-random color, making individual triangles easy to identify during testing. The GUI provides toggles for bounding-box debugging, depth buffering, depth-map visualization, backface culling, the Top-Left Fill Rule, and a wireframe overlay.

Part 1: Bounding Box Rasterization

After projection, the minimum and maximum screen-space x and y values of the three triangle vertices define an axis-aligned rectangle. The rectangle is clamped to the drawable viewport so that rasterization never writes outside the framebuffer or over the GUI area.

```
minimum_x = floor(min(v0.x, v1.x, v2.x));
maximum_x = ceil(max(v0.x, v1.x, v2.x));
minimum_y = floor(min(v0.y, v1.y, v2.y));
maximum_y = ceil(max(v0.y, v1.y, v2.y));
```

The debug mode fills every pixel in this rectangle without performing an inside-triangle test. Since every face has a different color, the result is a block-like collection of overlapping colored rectangles. This mode verifies that projected coordinates and screen bounds are correct before implementing the more precise triangle test.

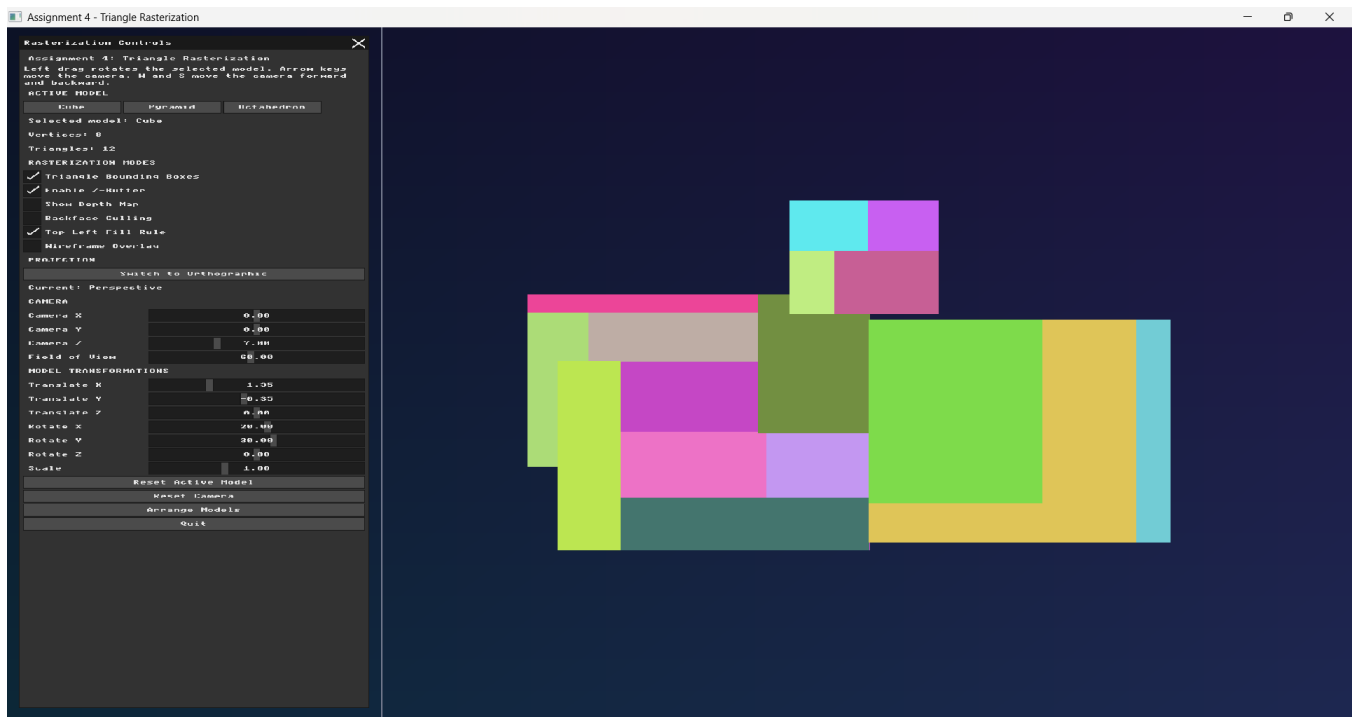


Figure 1. Triangle bounding-box rasterization using a different color for each projected face.

Part 2: Barycentric Triangle Filling

For every pixel center inside the triangle bounding box, three edge functions are evaluated. Their values are converted into barycentric weights α , β , and γ . A pixel belongs to the triangle when it lies on the accepted side of all three edges. The weights also satisfy $\alpha + \beta + \gamma = 1$, so they can be used to interpolate values defined at the vertices.

```

area = edge(v0, v1, v2)
alpha = edge(v1, v2, pixel) / area
beta = edge(v2, v0, pixel) / area
gamma = edge(v0, v1, pixel) / area

```

The pixel is sampled at its center, $(x + 0.5, y + 0.5)$, rather than at the top-left corner. Degenerate triangles with near-zero area are skipped. If the projected orientation is negative, two vertices are swapped so that the inclusion logic uses a consistent winding direction.

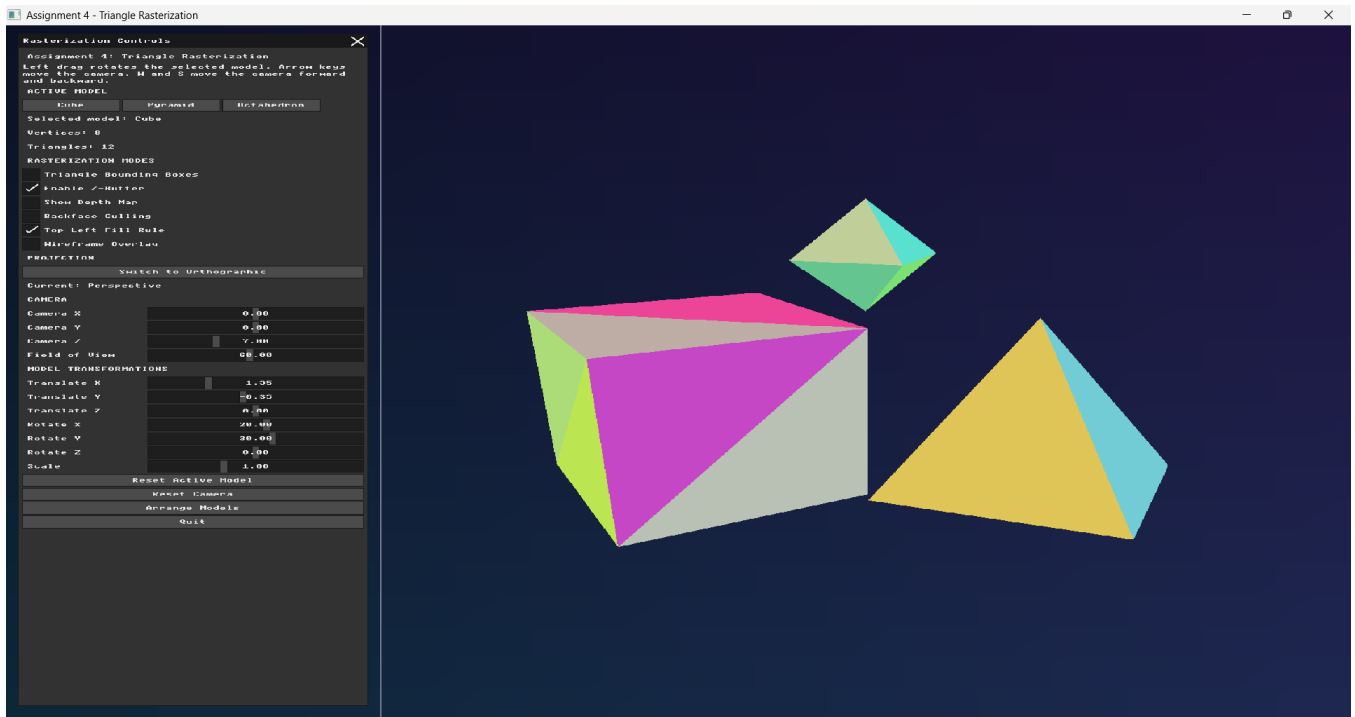


Figure 2. Solid meshes rasterized using barycentric coordinates and depth buffering.

Wireframe Overlay

A separate debugging toggle redraws the three projected edges after the solid face has been filled. The overlay exposes the original mesh topology and confirms that each visible polygon is composed of triangular faces.

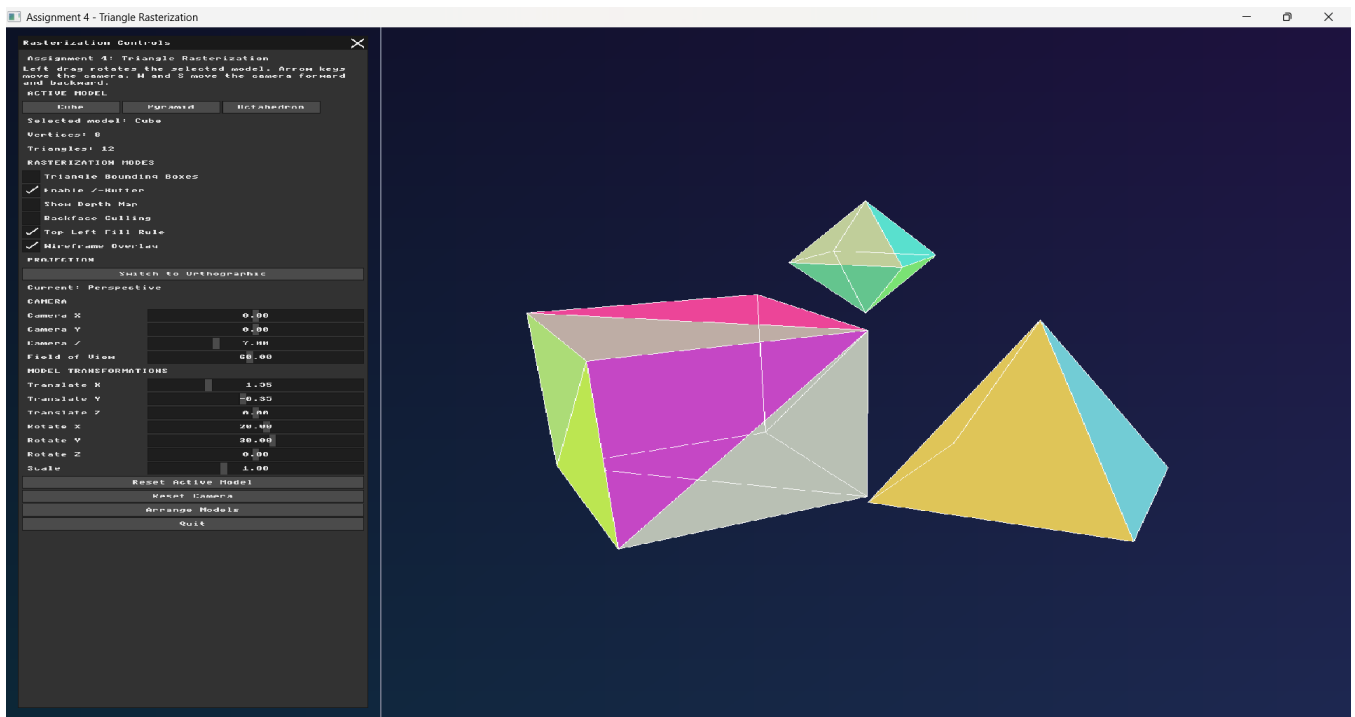


Figure 3. Solid triangle rasterization with a wireframe overlay showing the mesh topology.

Part 3: Z-Buffer Algorithm

A floating-point array with the same width and height as the color framebuffer is used as the depth buffer. At the beginning of every frame, all entries are initialized to positive infinity. Infinity represents a pixel at which no geometry has yet been drawn.

```
static float g_z_buffer[WIDTH * HEIGHT];  
for (float &depth : g_z_buffer) {  
    depth = std::numeric_limits<float>::infinity();  
}
```

The normalized depth of a candidate pixel is interpolated using the same barycentric weights used for the inclusion test:

$$\text{depth} = \alpha * v_0.z + \beta * v_1.z + \gamma * v_2.z;$$

The color is written only when this depth is smaller than the value already stored at the same pixel. When the test succeeds, both the color buffer and the Z-buffer are updated. This per-pixel comparison removes the ordering artifacts that occur when triangles are drawn only according to their processing order.

Depth Map Visualization

To visualize the Z-buffer, the minimum and maximum finite depth values in the scene are found and mapped to the grayscale range 0-255. Background pixels that still contain infinity are drawn black. The current implementation displays closer visible values as brighter shades and farther values as darker shades.

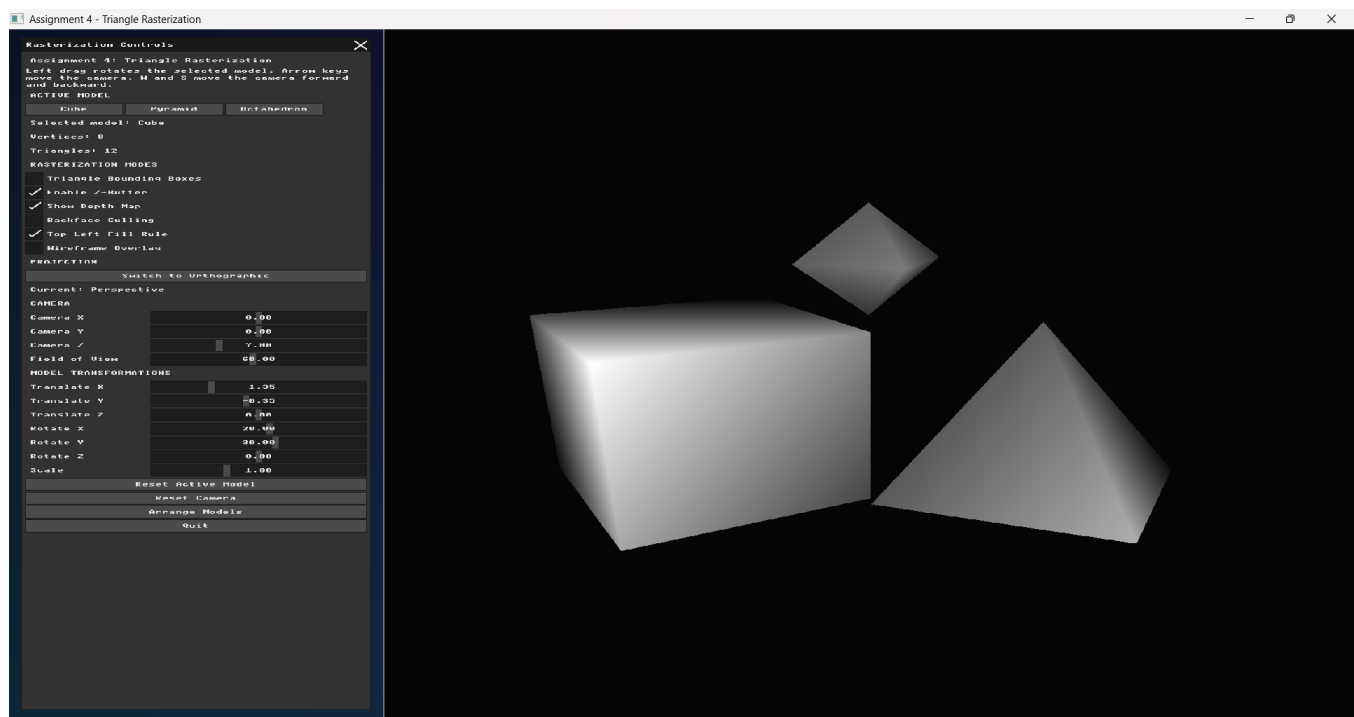


Figure 4. Grayscale visualization of the Z-buffer, where pixel intensity represents scene depth.

Part 4: Pair Programming Extensions

Backface Culling

Backface culling is performed before the expensive pixel loop. The three projected vertices also retain their view-space positions. Two triangle edges are crossed to calculate the face normal, and the triangle center defines a vector toward the camera. Their dot product determines whether the triangle faces the camera.

```
normal = cross(v1_view - v0_view, v2_view - v0_view);
view_direction = -triangle_center;
if (dot(normal, view_direction) <= 0) {
    discard triangle;
}
```

When culling is enabled, back-facing triangles are discarded before their screen bounding boxes and barycentric values are processed. Closed models appear visually similar because the Z-buffer would hide these surfaces, but fewer triangles enter the rasterization loop.

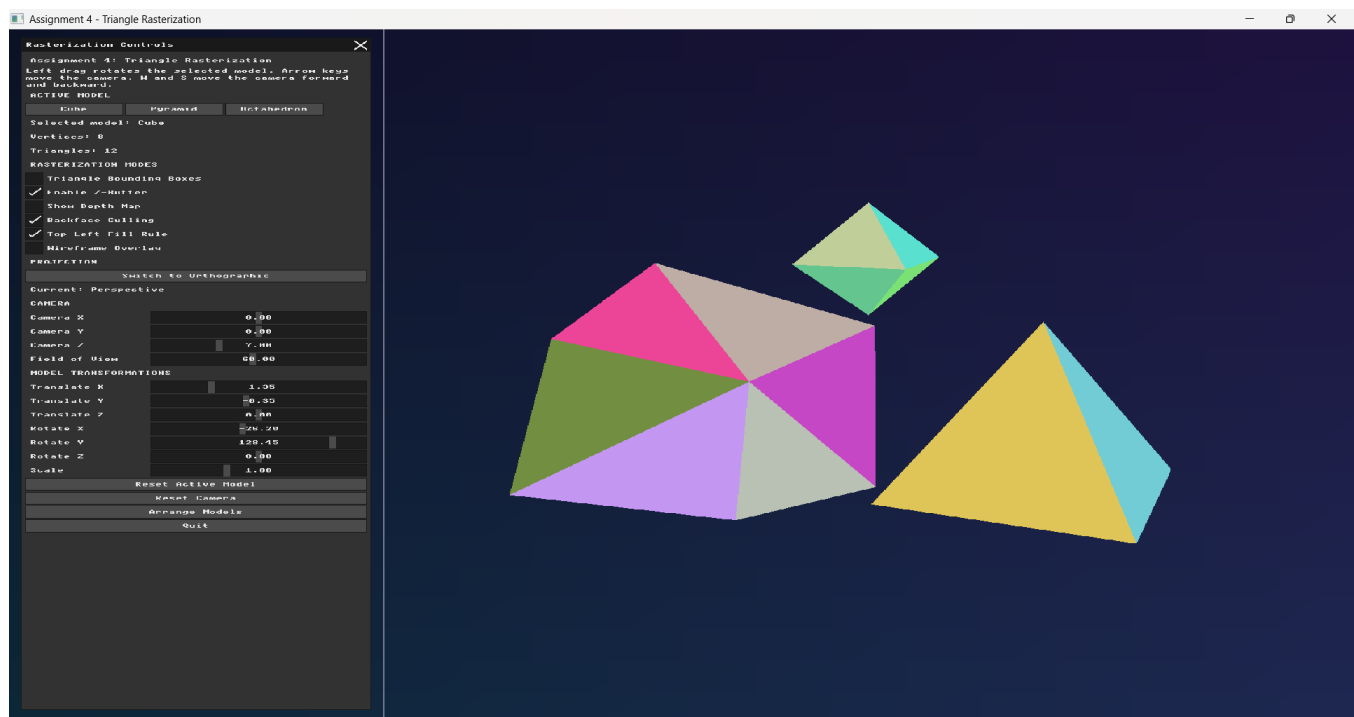


Figure 5. Rasterized scene with backface culling enabled, discarding triangles facing away from the camera.

Sub-pixel Precision and Top-Left Fill Rule

Adjacent triangles share edges. If all zero-valued edge tests are accepted by both triangles, the shared pixels may be drawn twice. If all are rejected, small cracks can appear. The Top-Left Fill Rule resolves the tie consistently: pixels exactly on an edge are accepted only when that edge is classified as a top or left edge. Therefore, exactly one of two adjacent triangles owns each shared-edge pixel.

```
top_left = (dy < 0) || (abs(dy) <= EPSILON && dx > 0);
inside = edge_value > EPSILON ||
        (abs(edge_value) <= EPSILON && top_left);
```

Interaction and Testing

The program was configured and built in CLion with CMake, MiniFB, MicroUI, and GLM 1.0.1. The following behaviors were verified:

- The cube, pyramid, and octahedron OBJ meshes load and display the expected vertex and triangle counts.
- Triangle bounding boxes are clamped to the rendering viewport.
- Barycentric tests fill the projected triangle interiors with solid colors.
- Enabling the Z-buffer removes incorrect overlap caused by triangle draw order.
- The depth-map mode converts stored depth values into visible grayscale intensities.
- Backface Culling can be enabled and disabled from the GUI.
- The Top-Left Fill Rule produces seamless shared edges.
- The wireframe overlay reveals triangle boundaries over the solid geometry.
- Perspective and orthographic projection modes remain available.
- The selected model can be rotated by dragging and adjusted using GUI controls.

Conclusion

This assignment completed the transition from wireframe rendering to solid software rasterization. The renderer now projects triangular geometry, restricts work to screen-space bounding boxes, identifies covered pixels with barycentric coordinates, interpolates depth, and resolves visibility using a Z-buffer. The depth-map and wireframe modes make the internal rasterization data observable and easier to debug.

The pair-programming extensions further improve correctness and efficiency. Backface Culling rejects triangles that face away from the camera before rasterization, while the Top-Left Fill Rule gives shared edges a deterministic owner and prevents cracks or duplicate coverage. Together, these components implement the central stages of a basic graphics pipeline entirely in software.